

Middleware: Comprehensive Exam Notes

Compiled from Course Lecture Slides

Contents

1	Introduction to Middleware	2
1.1	What is Middleware?	2
1.2	Services Provided by Middleware	2
1.3	Middleware Dimensions (Design Trade-offs)	2
2	Part I: Remote Procedure Call (RPC)	2
2.1	Overview of RPC	2
2.2	Properties of RPC	3
2.3	Failure Modes of RPC	3
2.4	Disadvantages of RPC	3
3	Part II: Object-Oriented Middleware (OOM)	3
3.1	Overview of OOM	3
3.2	Properties of OOM	4
3.3	Java Remote Method Invocation (RMI)	4
3.4	CORBA – Common Object Request Broker Architecture	4
3.5	CORBA IDL	4
3.6	CORBA Services (Selection)	5
3.7	Disadvantages of OOM	5
3.8	Reflective Middleware	5
4	Part III: Message-Oriented Middleware (MOM)	5
4.1	Overview of MOM	5
4.2	Properties of MOM	6
4.3	IBM MQSeries	6
4.4	Java Message Service (JMS)	6
4.5	Web Services	6
4.6	Disadvantages of MOM	7
5	Part IV: Event-Based Middleware (Publish/Subscribe)	7
5.1	Overview	7
5.2	Topic-Based and Content-Based Pub/Sub	7
5.3	Properties of Publish/Subscribe	8
5.4	Composite Event Detection (CED)	8

6	Middleware Categories (from Business Context)	8
6.1	Business Drivers for Middleware	8
6.2	Definition of Middleware (Business Perspective)	8
6.3	Three Categories of Middleware	8
6.4	Comparison: Tight vs. Loose Coupling	9
7	Summary	9

Introduction to Middleware

What is Middleware?

- A layer between the **Operating System (OS)** and **distributed applications**.
- Hides the complexity and heterogeneity of distributed systems.
- Bridges the gap between low-level OS communications and programming language abstractions.
- Provides a **common programming abstraction and infrastructure** for distributed applications.

Layered architecture (bottom to top):

Layer	Examples
Distributed Applications	(top layer)
Middleware	Remote calls, object invocation, messages
Operating System Comms	Sockets, IP, TCP, UDP
Network	Packets, bits

Services Provided by Middleware

Middleware provides support for (some of):

- Naming, Location, Service discovery, Replication
- Protocol handling, Communication faults, QoS
- Synchronisation, Concurrency, Transactions, Storage
- Access control, Authentication

Middleware Dimensions (Design Trade-offs)

Option A	vs.	Option B
Request/Reply	vs.	Asynchronous Messaging
Language-specific	vs.	Language-independent
Proprietary	vs.	Standards-based
Small-scale	vs.	Large-scale
Tightly-coupled	vs.	Loosely-coupled components

Part I: Remote Procedure Call (RPC)

Overview of RPC

- **Masks** remote function calls as being **local** to the programmer.
- Uses a **client/server model**.
- The request/reply paradigm is usually implemented with **message passing** in the RPC service.
- Requires **marshalling** of function parameters and return values.

RPC Call sequence (steps):

1. Marshal arguments

2. Generate call ID
3. Start timer (client side)
4. (Message sent) → Unmarshal on server
5. Record ID (server side)
6. Execute remote function `fun(...)`
7. Marshal result; set timer (server side)
8. (Reply sent) → Unmarshal result (client side)
9. Acknowledge

Properties of RPC

- **Language-level pattern of function call** – easy to understand for programmers.
- **Synchronous request/reply interaction:**
 - Natural from a programming language point-of-view.
 - Matches replies to requests.
 - Built-in synchronisation.
- **Distribution transparency** (no-failure case): hides the complexity of a distributed system.
- **Various reliability guarantees:** deals with some distributed systems aspects of failure.

Failure Modes of RPC

RPC invocation semantics in the light of: network and/or server congestion, client/network/server failure (independent failure modes).

- **Maybe or at most once** (RPC system tries once):
 - Error return – programmer may retry.
- **Exactly once** (RPC system retries a few times):
 - Hard error return – some failure most likely.
 - Note: “exactly once” *cannot* be guaranteed.

Disadvantages of RPC

- **Synchronous request/reply interaction:**
 - Tight coupling between client and server.
 - May block for a long time.
 - Leads to multi-threaded programming (e.g., `fork/join` pattern).
- **Distribution Transparency:** Not possible to mask all problems.
- **Lacks notion of services:** Programmer not interested in server but in service.
- **RPC paradigm is not object-oriented:** Invokes functions on servers as opposed to methods on objects.

Part II: Object-Oriented Middleware (OOM)

Overview of OOM

- **Objects** can be *local* or *remote*.

- **Object references** can be *local* or *remote*.
- Remote objects have visible **remote interfaces**.
- Masks remote objects as being local using **proxy objects**.
- Supports **remote method invocation**.

Architecture: Local object A interacts with a proxy of object B through an Object Request Broker (ORB) / object manager on both sides; on the remote side, a skeleton dispatches calls to the actual remote object B.

Properties of OOM

- **Support for object-oriented programming model:** objects, methods, interfaces, encapsulation, exceptions.
- **Location Transparency:** mapping object references to locations.
- **Synchronous request/reply interaction:** same as RPC.
- **Services:** provides service abstractions.

Java Remote Method Invocation (RMI)

- Implements distributed objects in Java.
- Interface definition example:

```
public interface PrintServer extends Remote {  
    int print(Vector printJob) throws RemoteException;  
}
```

- RMI compiler creates **proxies** and **skeletons** automatically.
- **RMI registry** used for interface lookup.
- All system written in Java (**single-language system**).

CORBA – Common Object Request Broker Architecture

- Open standard by the OMG (Version 3.0).
- **Language- and platform-independent**.

Object Request Broker (ORB):

- Uses **General Inter-ORB Protocol (GIOP)** for communication.
- Uses **Interoperable Object References (IOR)** which contain object location.
- **CORBA Interface Definition Language (IDL):** stubs (proxies) and skeletons are created by the IDL compiler.
- Supports **dynamic remote method invocation**.

Interface Repository: For querying existing remote interfaces.

Implementation Repository: For activating remote objects on demand.

CORBA IDL

- Defines **language-independent remote interfaces**.
- Language mappings to C++, Java, Smalltalk, etc.; translation by IDL compiler.
- **Type system:**

- *Basic types*: long (32-bit), long long (64-bit), short, float, char, boolean, octet, any, ...
- *Constructed types*: struct, union, sequence, array, enum
- *Objects*: common super type `Object`

```
typedef sequence<string> Files;  
interface PrintServer : Server {  
    void print(in Files printJob);  
};
```

- **Parameter passing**: in, out, inout; basic & constructed types passed by value; objects passed by reference.

CORBA Services (Selection)

- **Naming Service**: Names → remote object references.
- **Trading Service**: Attributes (properties) → remote object references.
- **Persistent Object Service**: Implementation of persistent CORBA objects.
- **Transaction Service**: Making object invocation part of transactions.
- **Event Service and Notification Service**: Asynchronous communication based on messaging (cf. MOM); not integrated programming model with general IDL messages.

Disadvantages of OOM

- **Synchronous request/reply interaction only**:
 - CORBA `oneway` semantics added; Asynchronous Method Invocation (AMI) added – but implementations may not be loosely coupled.
- **Distributed garbage collection**: Difficulty releasing memory for unused remote objects.
- **OOM rather static and heavy-weight**: Bad for ubiquitous systems and embedded devices.

Reflective Middleware

- Flexible middleware (OOM) for **mobile and context-aware applications** – adaptation to context through monitoring and substitution of components.
- **Interfaces for reflection**: Objects can inspect middleware behaviour.
- **Interfaces for customisability**:
 - Dynamic reconfiguration depending on environment.
 - Different protocols, QoS, ...
 - E.g., use different marshalling strategy over unreliable wireless link.

Part III: Message-Oriented Middleware (MOM)

Overview of MOM

- Communication using **messages**.
- Messages stored in **message queues**.

- Optional **message server** decouples client and server.
- Various assumptions about **message content**.

Properties of MOM

- **Asynchronous interaction:**
 - Client and server are only **loosely coupled**.
 - Messages are queued.
 - Good for application integration.
- **Support for reliable delivery service:** Queues kept in persistent storage.
- **Processing of messages by intermediate message server:** Filtering, transforming, logging; networks of message servers.
- **Natural for database integration.**

IBM MQSeries

- One-to-one reliable message passing using queues.
- Supports persistent and non-persistent messages; message priorities; message notification.
- **Queue Managers:** Responsible for queues; transfer messages from input to output queues; keep routing tables.
- **Message Channels:** Reliable connections between queue managers.
- **Messaging API:**

Call	Description
MQopen	Open a queue
MQclose	Close a queue
MQput	Put message into opened queue
MQget	Get message from local queue

Java Message Service (JMS)

- **API specification** to access MOM implementations.
- Two modes of operation:
 - **Point-to-point:** One-to-one communication using queues.
 - **Publish/Subscribe:** cf. Event-Based Middleware.
- **JMS Server** implements JMS API; JMS Clients connect to JMS servers.
- Java objects can be serialised to JMS messages.

Web Services

- Use **well-known web standards** for distributed computing.

Communication:

- Message content expressed in **XML**.
- **Simple Object Access Protocol (SOAP):** Lightweight protocol for sync/async communication; supports internet transports (http, smtp, ...); uses XML Schema for marshalling types.

Service Description:

- **Web Services Description Language (WSDL):** Interface description for web services; says how to use a web service.

Service Discovery:

- **Universal Description Discovery and Integration (UDDI):** Directory with web service descriptions in WSDL; helps find the right web service; exports SOAP API for access.

Properties of Web Services:

- Language-independent and open standard.
- **SOAP** combines best of both worlds: synchronous request/reply like OOM and asynchronous messaging like MOM.

Disadvantages of MOM

- **Poor programming abstraction** (but has evolved):
 - Rather low-level (cf. Packets).
 - Request/reply more difficult to achieve.
 - Results in multi-threaded code.
- **Message formats unknown to middleware:** No type checking (JMS partially addresses this).
- **Queue abstraction only gives one-to-one communication:** Limits scalability (JMS pub/sub partially addresses this).

Part IV: Event-Based Middleware (Publish/Subscribe)

Overview

- **Publishers** *publish* events (messages).
- **Subscribers** express interest in events with *subscriptions*.
- **Event Service** *notifies* interested subscribers of published events.
- Events can have arbitrary content or name/value pairs.

Topic-Based and Content-Based Pub/Sub**Topic-Based Publish/Subscribe:**

- Publishers publish events belonging to a **topic** or **subject**.
- Subscribers subscribe to a **topic**.
- Example: `subscribe(PrintJobFinishedTopic, ...)`

(Topic and) Content-Based Publish/Subscribe:

- Publishers publish events belonging to topics.
- Subscribers provide a **filter** based on *content* of events.
- Example: `subscribe(type=printjobfinshed, printer='aspen', ...)`

Properties of Publish/Subscribe

- **Asynchronous communication:** Publishers and subscribers are loosely coupled.
- **Many-to-many interaction** between publishers and subscribers:
 - Scalable scheme for large-scale systems.
 - Publishers do not need to know subscribers.
- **Content-based pub/sub is very expressive:** Filtered information delivered only to interested parties.

Composite Event Detection (CED)

- Content-based pub/sub is *not expressive enough* when there are potentially thousands of event types and subscribers are interested in **patterns of events**.
- Example event pattern: `PrinterOutOfPaperEvent` OR `PrinterOutOfTonerEvent`.
- **Composite Event Detectors (CED):** Subscribe to primitive events and publish composite events.
- CEDs can be chained – a CED can subscribe to events published by another CED.

Middleware Categories (from Business Context)

Business Drivers for Middleware

- 90% of IT budgets are spent maintaining and enhancing existing systems.
- Business Process Reengineering (BPR) techniques help organisations achieve competitive advantage through improvements in quality, customer service, and costs.
- BPR must be aligned with technology strategy to be effective; organisations must use technology to enable business change defined by BPR.
- Technology personnel costs rise faster than inflation; professionals change jobs every 3–5 years $\Rightarrow$ need for continuous learning programs.
- The application backlog is severe: 80–90% of traditional MIS staff time is devoted to maintenance.
- Need for new mission-critical systems built quickly with highly productive teams partnered with end-users.
- Need for platform migration and reengineering of existing systems.
- Need for a common interface across platforms.
- Increase in application development by users.

Definition of Middleware (Business Perspective)

- Easiest way to integrate heterogeneous components is *not* to recreate them as homogeneous elements, but to provide a **communication layer** despite their differences.
- Allows software components (applications, EJBs, servlets, etc.) developed independently on different networked platforms to **interact with one another**.

Three Categories of Middleware

1. **Remote Procedure Call (RPC)-based middleware:** Allows procedures in one application to call procedures in remote applications as if they were local. Imple-

ments a linking mechanism that locates remote procedures transparently. Now also includes object-based components.

2. **Object Request Broker (ORB)-based middleware:** Enables an application's objects to be distributed and shared across heterogeneous networks.
3. **Message Oriented Middleware (MOM)-based middleware:** Allows distributed applications to communicate and exchange data by sending and receiving messages.

Comparison: Tight vs. Loose Coupling

- **RPC- and ORB-based** systems create **tightly-coupled** components: the caller must wait for the called procedure to return before it can do anything else (*synchronous* messaging).
- **MOM-based** systems allow **loosely-coupled** components via **asynchronous** exchange of messages.
- In synchronous systems, middleware acts partly as a *super-linker*: locates the called procedure on the network, passes function/method parameters, and returns results.

Summary

- Middleware is an important **abstraction for building distributed systems**.
- Four paradigms covered:
 1. Remote Procedure Call
 2. Object-Oriented Middleware
 3. Message-Oriented Middleware
 4. Event-Based Middleware
- Key themes: Synchronous vs. asynchronous communication; scalability and many-to-many communication; language integration; support for ubiquitous and mobile systems.